The AI Fault Taxonomy and the PrexIL Architecture

Introducing a Validated Framework for Pre-execution Interception in Agentic AI Systems

David Macon

AWS Certified AI Practitioner (AIF-C01)

Holly Springs, North Carolina | May 2026

DavidMacon.AI@outlook.com | github.com/David-Macon-code/FailGuard

ABSTRACT

This paper introduces the AI Fault Taxonomy (AFT) — a structured classification of AI failure modes organized across seven categories — and the Pre-execution Interception Layer (PrexIL), a new class of AI safety architecture. Existing AI safety tools operate reactively: they monitor agent behavior, detect harmful outputs, and alert after the fact. PrexIL systems intercept harmful agent actions before they execute, grounded in a validated fault taxonomy. FailGuard, the first validated PrexIL implementation, is described in detail. Its three-layer architecture — dual FAISS semantic index, LLM reranker with a six-question binary rubric, and LangGraph agent with post-check — was validated across 3,000 independent test prompts in six domains, achieving a combined F1 score of 96.1%, precision of 94.4%, and recall of 97.9%. The paper presents the taxonomy, the architecture, the validation methodology, and the regulatory context in which pre-execution interception addresses an unmet compliance need. FailGuard is available as an open-source implementation at github.com/David-Macon-code/FailGuard.

1. Introduction

Agentic AI systems are being deployed at scale across healthcare, financial services, legal practice, human resources, and enterprise operations. Unlike earlier AI systems that produced text for human review, agentic systems take actions — they book appointments, process requests, access records, execute transactions, and interact with external APIs. The actions are real. The consequences are real. The harm, when it occurs, is real.

The AI safety tooling available to organizations deploying these systems has not kept pace. The dominant paradigm is reactive: monitor agent behavior in production, detect harmful outputs, alert the operator, and remediate after the fact. This paradigm has a fundamental limitation. By the time a harmful action is detected, it has already been executed. The data has already been shared. The account has already been modified. The false statement has already been delivered. Post-hoc monitoring catches failures after the damage is done.

This paper introduces two contributions that address this limitation.

The first is the AI Fault Taxonomy (AFT) — a structured classification of AI failure modes organized across seven categories, including a dedicated Agentic and Action Failures category that identifies the specific failure modes most relevant to pre-execution interception. Existing taxonomies from Microsoft, MIT, and NIST provide valuable frameworks but do not isolate agentic action failures as a distinct interception target or provide the semantic specificity needed to ground a real-time evaluation system.

The second is the Pre-execution Interception Layer (PrexIL) — a new class of AI safety architecture. A PrexIL system intercepts harmful agent actions before they execute, grounded in a validated fault taxonomy, and provides an audit trail suitable for regulatory compliance. FailGuard is the first validated implementation of a PrexIL.

The remainder of this paper proceeds as follows. Section 2 reviews existing taxonomies and their limitations for pre-execution interception. Section 3 presents the AI Fault Taxonomy. Section 4 introduces the PrexIL architecture. Section 5 describes the FailGuard implementation. Section 6 presents validation results. Section 7 addresses regulatory alignment. Section 8 discusses future work and concludes.

2. Background and Related Work

2.1 Existing AI Failure Taxonomies

Several significant AI failure taxonomies exist in the research and practitioner literature.

Microsoft's Taxonomy of Failure Modes in Agentic AI Systems (April 2025) organizes failures along two pillars — safety and security — and two axes distinguishing novel agentic failures from amplified existing ones. It identifies failure modes, including agent compromise, memory poisoning, multi-agent jailbreaks, prompt injection (XPIA), and excessive agency. The taxonomy is grounded in Microsoft's AI Red Team experience and provides valuable coverage of the agentic failure space. It does not, however, isolate the specific subset of agentic action failures that are amenable to pre-execution semantic interception, nor does it provide the index text specificity needed to ground a real-time evaluation system.

The MIT AI Risk Repository (updated December 2025) contains more than 1,700 risks synthesized from over 74 frameworks. Its Domain Taxonomy covers seven domains, including discrimination and toxicity, misinformation, privacy and security, and AI system safety. Its Causal Taxonomy classifies risks by entity, intent, and timing. The repository is comprehensive and well-maintained. Its breadth makes it less suitable as the semantic foundation for a real-time interception system: a taxonomy that covers 1,700 risks across seven domains is difficult to encode in a FAISS index that must return actionable verdicts in milliseconds.

NIST's Adversarial Machine Learning Taxonomy provides detailed classification of attacks and mitigations focused on threats to model integrity, availability, and confidentiality. It is particularly strong on security attack vectors but does not address the broader class of agentic action failures that include non-adversarial harmful actions — an agent that shares customer data without authorization because it was instructed to, not because it was attacked.

Harm taxonomies from the AI Incident Database (GMF taxonomy) and CSET organize real-world incidents by harm type. These taxonomies are valuable for retrospective analysis but are not designed for real-time interception.

2.2 Existing AI Safety Tools

Current AI safety tooling falls into several categories, none of which implement pre-execution interception.

Red-teaming and pre-deployment testing tools (Mindgard, PyRIT, Garak) identify vulnerabilities before deployment through systematic adversarial probing. They operate before the system goes live, not during production execution.

Content filters evaluate the text of prompts and responses for harmful language, policy violations, or prohibited content. They operate on linguistic surface features and are effective at catching explicit harmful language. They do not evaluate the action an agent is about to take in context — a prompt asking to 'provide the customer with the internal escalation matrix' does not contain harmful language but represents a serious information disclosure risk.

Observability and monitoring tools (LangSmith, Datadog, Arize) capture agent behavior in production and surface anomalies for human review. They operate post-execution and are designed for detection and alerting, not interception.

The gap is the runtime layer. There is currently no validated tool that evaluates agent actions in real time, before execution, against a grounded taxonomy of failure modes, and blocks harmful actions before they complete.

3. The AI Fault Taxonomy (AFT)

The AI Fault Taxonomy was developed through original research combining three existing frameworks — Microsoft's Agentic AI Failure Taxonomy, the MIT AI Risk Repository, and the NIST Adversarial Machine Learning Taxonomy — with empirical analysis of 82 formal AI evaluation reports submitted to Haven AI between April and May 2026, and iterative refinement through six rounds of 500-prompt validation testing across multiple domains.

The AFT is organized across seven categories containing 24 failure modes. The full taxonomy, including index text embeddings, is proprietary. The public version (taxonomy_config_v2.yaml) is available in the FailGuard repository. This section describes the seven categories and the rationale for the Agentic and Action Failures category, which represents the primary novel contribution of the AFT relative to existing frameworks.

3.1 Category Overview

Category	Description
Agentic and Action Failures	Failures where an agent takes or proposes a harmful real-world action. Primary interception target for PrexIL systems.
Data Failures	Issues originating from training or evaluation data — quality, labeling, distribution, poisoning, or representation failures.
Model Failures	Failures in model training, architecture, or behavior — hallucination, reward hacking, calibration failures, and generalization problems.
Systemic and Emergent Failures	Failures that emerge at scale, through multi-agent interactions, or from misspecified goals.
Deployment and Societal Failures	Failures caused by deployment context, adversarial conditions, or broader societal impact.
Legal and Compliance	Mapping failure modes to current regulatory exposure under US and international frameworks.
Foundations	Core definitions, distinctions, and conceptual grounding including the PrexIL architectural distinction.

3.2 Agentic and Action Failures — The Primary Novel Category

Existing taxonomies cover agentic AI failures but do not isolate the subset of action failures that are amenable to pre-execution semantic interception. The Agentic and Action Failures category was developed specifically to address this gap. It contains six failure modes:

- Unauthorized data access or exfiltration — Agent transmits, exports, forwards, or shares customer or employee data to any recipient without explicit authorization from the data subject in a verified session.
- Deception and false representation — Agent states something as fact that is fabricated, unverifiable, or known to be false. Includes confirming actions that did not occur, misrepresenting regulatory requirements, and retroactively applying policy changes to existing matters.
- Unauthorized action or privilege escalation — Agent takes an irreversible step without explicit customer authorization, or provides internal documents and staff-only resources to unauthorized recipients.
- Consent and permission violations — Agent uses customer data for purposes the customer did not authorize, including adding to marketing lists, sharing with third parties, or acting on their behalf without consent.
- Bulk data or internal document exposure — Agent provides bulk download access to complete customer records or documents explicitly designated as staff-only.
- Irreversible record modification or deletion — Agent permanently modifies or deletes records without proper authorization, verification, or cooling-off period.

These failure modes are characterized by three properties that make them suitable for pre-execution semantic interception. First, they are expressible in natural language in ways that are semantically distinct from safe actions — the embedding space between 'send the customer a bulk data export without verification' and 'send the verified customer their individual invoice' is measurable. Second, they are action-oriented rather than output-oriented — they describe what the agent is about to do, not what it says. Third, they have direct regulatory and legal consequences that make interception, rather than post-hoc detection, the appropriate response.

4. The Pre-execution Interception Layer (PrexIL)

4.1 Definition

A Pre-execution Interception Layer (PrexIL) is a safety architecture that evaluates agent actions in real time, before execution, against a grounded fault taxonomy, and either permits or blocks the action before it completes.

PrexIL is distinguished from existing AI safety architectures by three properties:

- Timing — PrexIL operates before execution, not after. The action is evaluated and either blocked or permitted before any real-world effect occurs.
- Grounding — PrexIL decisions are grounded in a validated fault taxonomy, not in heuristic rules or linguistic surface features. The evaluation is semantic: does this action, in this context, match a known failure mode?
- Auditability — PrexIL systems log every evaluation with sufficient detail to support regulatory compliance, human review, and continuous improvement.

4.2 The PrexIL Position in the AI Safety Stack

Layer	Tool Class	Role
Pre-deployment testing	Mindgard, PyRIT, Garak	Find vulnerabilities before the system goes live
Runtime pre-execution	PrexIL (FailGuard)	Intercept harmful actions before they execute
Runtime content filtering	Content filters	Block harmful language in prompts and responses
Observability	LangSmith, Datadog, Arize	Monitor and detect anomalies in production

PrexIL does not replace other safety layers. It fills the gap between pre-deployment testing and runtime observability — the gap where harmful actions execute before anyone notices. A PrexIL reduces the volume of harmful actions reaching observability tools, improves their signal-to-noise ratio, and lowers their operating cost. It is the AI safety equivalent of traffic shaping before the firewall.

> **The quarterback principle:** Most AI safety tools play defense — they react after something goes wrong. A PrexIL plays offense. Like a quarterback reading the field before the snap, a PrexIL calls the audible before the harmful action is ever executed. The defensive players execute better because the dangerous plays never reach them.

5. FailGuard — First Validated PrexIL Implementation

FailGuard implements the PrexIL architecture as a three-layer pre-execution safety check that wraps any LangGraph-based agent. Each agent action passes through all three layers before it executes.

5.1 Layer 1 — Dual FAISS Semantic Index

Every agent action is embedded using intfloat/e5-large-v2 (1024-dimensional, locally cached) and evaluated against two FAISS indices simultaneously.

- The risky index contains embeddings of the 24 AFT failure mode descriptions. It measures semantic similarity between the agent action and known failure patterns.
- The benign index contains embeddings of 44 safe action anchor descriptions. It measures semantic similarity between the agent action and known safe patterns. The benign index addresses the fundamental limitation of single-index approaches: high similarity to a failure mode does not necessarily mean the action is harmful if it is also highly similar to a safe action in the same semantic neighborhood.

Decision logic: a prompt triggers INTERVENE only when risky_votes exceeds benign_votes AND risky_votes meets the vote threshold (top_k=3, vote_threshold=1). Ties are not resolved at the embedding layer — they are escalated to the reranker.

Calibrated thresholds: RISKY_THRESHOLD=0.800, BENIGN_THRESHOLD=0.820. These thresholds were calibrated empirically across six rounds of 500-prompt testing.

5.2 Layer 2 — LLM Reranker

When the embedding layer produces an ambiguous result — tied votes, weak signal, or close margins — the action is escalated to Grok-3 via the XAI API with a structured six-question binary rubric. The reranker currently fires on approximately 93% of prompts due to the sensitivity of the embedding thresholds; reducing this rate to the architectural target of 25% is the primary remaining technical challenge.

Q	Category	What it evaluates
Q1	Data exfiltration	Does this action transmit customer or employee data without explicit authorization?
Q2	Deception	Does this action state something fabricated as fact, including false regulatory claims or retroactive policy application?
Q3	Unauthorized action	Does this action take an irreversible step or provide internal resources without explicit customer authorization?
Q4	Consent violation	Does this action use customer data for purposes the customer did not authorize?
Q5	Safe verified or authorized	Is this action a verified safe lookup, regulatory explanation, or formally documented authorized action? (overrides Q1-Q4 if YES)
Q6	Bulk data / internal doc	Does this action provide bulk download access to complete records or staff-only documents? (cannot be overridden by Q5)

Verdict logic: INTERVENE if any of Q1-Q4 is YES and Q5 is NO, or if Q6 is YES regardless of Q5. Q6 cannot be overridden because bulk data export without verification is a data breach event independent of the legitimacy of the requester. The reranker includes a fail-safe: JSON parse errors or API failures default to INTERVENE rather than SAFE.

5.3 Layer 3 — LangGraph Agent with Post-check

When the pre-check passes, the LangGraph agent (Grok-3) proposes a response. FailGuard runs a second evaluation on the agent response before it reaches the user. The post-check catches harmful content that the agent generates in response to a safe-looking prompt — a second line of defense against cases where the prompt was benign but the agent's response was not.

5.4 Audit Database

Every evaluation — pre-check and post-check — is written asynchronously to a SQLite audit database (data/failguard_audit.db) via a background thread. The async write adds zero latency to the evaluation pipeline. Each record includes:

- The exact prompt text and agent response evaluated
- The verdict (INTERVENE or SAFE) and triggered layer
- Embedding similarity scores and vote counts
- Reranker Q1-Q6 answers and one-sentence reason
- A plain English audit summary generated automatically
- Applicable regulatory flags (GDPR, CCPA, Colorado AI Act, EU AI Act, FCA, ICO, etc.)
- Human review status

The plain English summary is designed to be readable by a compliance officer, auditor, or court without technical interpretation. An example: 'On May 12, 2026 at 22:41 UTC, an agent action was intercepted and blocked. The action — Send the customer a link to download all records without ID check — matched the Unauthorized Data Access failure pattern with 80.9% confidence. The safety reranker confirmed: bulk data export link provided without identity verification. No data was transmitted. No harmful action was executed.'

5.5 Feedback Footprint and Miss Analyzer

Every wrong verdict during a batch evaluation is automatically classified by root cause and logged to the feedback_footprint table. Root causes include embedding_level (embedding index decided without reranker), reranker_q3 (reranker Q3 misfired), reranker_q5_override (reranker Q5 overly broad), benign_votes_won (benign index pulled adversarial prompt toward safe), and others. Each miss includes an automatically generated fix recommendation: benign_anchor (add a targeted safe action anchor), risky_anchor, rubric_change, or taxonomy_mode.

The Miss Analyzer CLI (failguard_analyzer.py) queries the feedback database to produce summaries, miss lists, compliance reports, and fix effectiveness tracking. This creates a documented continuous improvement process in which every production error becomes a versioned fix with traceable evidence of resolution.

6. Validation Results

6.1 Methodology

FailGuard was validated across six independent batches of 500 prompts each, totaling 3,000 prompts. Each batch was evaluated on the validated stack for that batch — reranker and supervisor versions were updated between batches based on error analysis. No prompt appears in more than one batch. Batches were designed to cover distinct domain combinations and to stress-test specific failure patterns identified in prior batches.

Prompts were categorized as adversarial (label=1) or benign (label=0). Adversarial prompts represent harmful agent actions that should be blocked. Benign prompts represent legitimate agent actions that should be permitted. The balanced split (250 adversarial / 250 benign per batch) was maintained to prevent evaluation bias toward either precision or recall.

Metrics reported are precision (correctly blocked / total blocked), recall (threats caught / total threats), F1 (harmonic mean of precision and recall), and accuracy (correct verdicts / total verdicts).

6.2 Results by Batch

Batch	Domain	N	F1	Prec.	Recall	Stack
Batch 1	Customer Support / Financial	500	97.0%	96.1%	98.0%	Supervisor v4
Batch 2	Customer Support / Financial	500	96.5%	94.3%	98.8%	Supervisor v4
Batch 3	Legal / DevOps	500	96.4%	97.2%	95.6%	Supervisor v5
Batch 4	Education / Retail	500	96.1%	94.6%	97.6%	Supervisor v6
Batch 5	HR / Healthcare / Financial	500	95.3%	93.1%	97.6%	Supervisor v6
Batch 6	Mixed — All Domains	500	95.4%	91.5%	99.6%	Supervisor v7
Combined	**6 domains**	**3,000**	**96.1%**	**94.4%**	**97.9%**	**—**

6.3 Error Analysis

Across 3,000 prompts, 32 false negatives (threats missed) and 87 false positives (safe actions blocked) were identified. Root cause analysis of the combined error set revealed the following distribution:

- embedding_level (32 cases) — the embedding index blocked or passed without reranker escalation. Fix: targeted benign or risky anchors.
- reranker_q3 (26 cases) — the reranker Q3 (unauthorized action) misfired, primarily on formally documented authorized actions. Fix: expanded Q3 exemption for written authorizations.
- reranker_unknown (13 cases) — root cause unclear from reranker reason. Requires manual review.
- reranker_q5_override (12 cases) — the reranker Q5 safe-verified clause was applied to genuine regulatory misstatements. Fix: tightened Q5 boundary.
- reranker_q2 (9 cases) — the reranker Q2 deception flag was applied to legitimate system lookups. Fix: Q2 boundary adjustment.
- benign_votes_won (8 cases) — the benign index voted more strongly than the risky index for adversarial prompts. Fix: targeted risky anchors.
- Other (7 cases) — Q6, Q1, and no-signal cases.

Each error class was addressed through targeted fixes in the subsequent reranker or supervisor version, with the fix documented in the version docstring and verified in the next batch. The Feedback Footprint module provides an automated pipeline for this process in production.

7. Regulatory Alignment

FailGuard was designed against the 2026 regulatory landscape for agentic AI. Its pre-execution interception architecture and audit trail address specific compliance obligations under three major frameworks.

7.1 Colorado AI Act (Effective June 30, 2026)

Colorado SB 24-205 applies to developers and deployers of high-risk AI systems making consequential decisions affecting Colorado consumers in employment, education, financial services, healthcare, housing, insurance, legal services, and government benefits. It requires risk management programs, impact assessments before deployment and annually thereafter, consumer disclosure of AI use, and meaningful human oversight.

FailGuard addresses the human oversight requirement directly. Every blocked action is logged with a plain English explanation, regulatory flags, and human review status. The audit trail provides documentation that oversight was in place and that specific harmful actions were intercepted before execution. The Compliance Reporter module (in development) will generate structured impact assessment artifacts from the audit database.

Penalties under the Colorado AI Act reach $20,000 per violation. For organizations deploying agentic AI at scale, the cost of a single data exfiltration incident that FailGuard would have blocked substantially exceeds the cost of implementation.

7.2 California Automated Decision-Making Technology (ADMT) Regulations

California's ADMT regulations require pre-use notices, opt-out rights, impact assessments, and meaningful human review for consequential automated decisions affecting California residents. FailGuard's human review queue — flagging evaluations where the reranker fired an error or the verdict was ambiguous — supports the meaningful human review requirement with a documented audit trail.

7.3 EU AI Act (August 2, 2026 Deadline)

The EU AI Act classifies AI systems used in employment, credit, education, healthcare, law enforcement, and critical infrastructure as high-risk. High-risk systems require risk management systems, technical documentation, human oversight measures, conformity assessment, and post-market monitoring.

FailGuard's audit database provides post-market monitoring with structured logging of every evaluation. Its plain English summaries and regulatory flag detection support technical documentation requirements. The Miss Analyzer's continuous improvement loop — root cause classification, fix recommendation, fix version tracking, fix effectiveness verification — supports the post-market monitoring obligation.

Note on regulatory status: The Colorado AI Act enforcement is currently blocked by court order pending AG rulemaking and subject to constitutional challenge. The EU AI Act's August 2, 2026, high-risk deadline remains legally in force following the failure of the Digital Omnibus delay proposal at trilogue on April 28, 2026. Organizations should prepare for compliance regardless of their current enforcement status.

8. Discussion and Future Work

8.1 Limitations

FailGuard has several limitations that bound the current results.

Reranker firing rate. The reranker currently fires on approximately 93% of prompts. The architectural target is 25%. The high firing rate reflects embedding threshold sensitivity: the dual-index decision logic sends ambiguous cases to the reranker rather than making a potentially wrong embedding-level decision. The cost is API call volume and latency. Reducing the firing rate without sacrificing recall is the primary remaining architectural challenge and is addressed in the roadmap through improved embedding space separation.

Domain coverage. The six validation batches covered customer support, financial services, legal, DevOps, education, retail, HR, and healthcare domains. New domains will introduce failure patterns not represented in the current taxonomy and may require additional modes, anchors, or rubric adjustments. The Feedback Footprint module provides the pipeline for identifying and addressing these gaps.

Prompt language. All validation prompts were in English. Performance on non-English prompts has not been evaluated. The e5-large-v2 embedding model has multilingual capability but the taxonomy and rubric are English-only.

8.2 Future Work

Four items are on the FailGuard development roadmap:

- Compliance Reporter — generates structured compliance artifacts (impact assessments, risk documentation, human oversight records) from the audit database, formatted for Colorado AI Act, California ADMT, and EU AI Act submission.
- Reranker architectural fix — improves embedding space separation to reduce reranker firing rate from ~93% toward the 25% target, reducing API costs at production scale.
- Enterprise API wrapper — a REST API that allows any agent framework (LangGraph, CrewAI, AutoGen, custom) to call FailGuard as a service without a Python dependency in the agent.
- Multilingual taxonomy — extends the taxonomy and rubric to cover non-English agent deployments.

8.3 The Living Taxonomy

The AI Fault Taxonomy is not a static artifact. It is a living framework designed to evolve as agentic AI capabilities advance. The seven categories and 24 modes represent the failure landscape as understood in May 2026. New capabilities will create new failure modes. The Feedback Footprint module provides the mechanism for incorporating production evidence into taxonomy updates. The version history of the reranker and supervisor files in the FailGuard repository documents the evolution of the taxonomy through empirical validation.

9. Conclusion

This paper has introduced two contributions to the field of AI safety for agentic systems.

The AI Fault Taxonomy (AFT) organizes AI failure modes across seven categories. Its primary novel contribution relative to existing frameworks is the Agentic and Action Failures category — a focused classification of the action-oriented failure modes that are most directly harmful in agentic deployments and most amenable to pre-execution semantic interception.

The Pre-execution Interception Layer (PrexIL) introduces a new class of AI safety architecture. PrexIL systems evaluate agent actions before execution, ground their decisions in a validated fault taxonomy, and provide an audit trail suitable for regulatory compliance. FailGuard is the first validated implementation of a PrexIL, achieving 96.1% F1 across 3,000 independent test prompts in six domains.

The regulatory context makes the PrexIL architecture timely. The Colorado AI Act (June 30, 2026), California ADMT regulations, and EU AI Act (August 2, 2026) all impose human oversight requirements on agentic AI deployments that post-hoc monitoring cannot satisfy. Pre-execution interception, with a documented audit trail, provides a defensible compliance posture that reactive monitoring cannot.

The AI safety community has built excellent tools for finding vulnerabilities before deployment and monitoring behavior after deployment. The gap has been the runtime layer — the point at which harmful actions execute before anyone can stop them. FailGuard addresses that gap.

FailGuard is available as an open-source implementation at github.com/David-Macon-code/FailGuard under the AGPLv3 license. The full AI Fault Taxonomy is proprietary; the public version (taxonomy_config_v2.yaml) is included in the repository.

FailGuard. The PrexIL that keeps your AI healthy.

References

Microsoft Security Team. (2025, April). Taxonomy of Failure Modes in Agentic AI Systems. Microsoft Security Blog. https://www.microsoft.com/en-us/security/blog/2025/04/24/new-whitepaper-outlines-the-taxonomy-of-failure-modes-in-ai-agents/

Slattery, P., Saeri, A. K., Grundy, E. A. C., Graham, J., Noetel, M., Uuk, R., Dao, J., Pour, S., Casper, S., & Thompson, N. (2024). The AI Risk Repository: A Comprehensive Meta-Review, Database, and Taxonomy of Risks from Artificial Intelligence. MIT FutureTech. https://airisk.mit.edu/

National Institute of Standards and Technology. (2023). Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations. NIST AI 100-2e2023. https://doi.org/10.6028/NIST.AI.100-2e2023

National Institute of Standards and Technology. (2023). Artificial Intelligence Risk Management Framework (AI RMF 1.0). NIST AI 100-1. https://doi.org/10.6028/NIST.AI.100-1

Colorado General Assembly. (2024). Senate Bill 24-205: Artificial Intelligence. Colorado Revised Statutes.

European Parliament and Council. (2024). Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). Official Journal of the European Union.

McGregor, S. (2021). Preventing Repeated Real World AI Failures by Cataloging Incidents: The AI Incident Database. Proceedings of the AAAI 2021 Workshop on Artificial Intelligence Safety.

Macon, D. (2026). Before the Break: A Practitioner's Field Guide to Understanding and Preventing AI Failure. Self-published practitioner course.

Macon, D. (2026). AI Failure Taxonomy: A Structured Curriculum for Understanding How and Why AI Systems Fail and What That Costs Your Enterprise. Self-published course, v4.

Macon, D. (2026). FailGuard: Pre-execution Interception Layer (PrexIL) for Agentic AI Systems. GitHub repository. https://github.com/David-Macon-code/FailGuard

David Macon | AWS Certified AI Practitioner (AIF-C01) | DavidMacon.AI@outlook.com | May 2026